Comp 533 - Assignment 7: Centralized Two-Phase Synchronous Consensus and Atomic Broadcast (Extra Credit)

Completion Date: Monday, April 28, 2025, 11:55 pm

No Late Submission!

The goal of this assignment is to introduce you to additional mechanisms for making stronger guarantees regarding proposal consensus. Specifically, you will be implementing atomic broadcast and centralized two-phase synchronous consensus using RMI. This means phase 1 communication between the relayer and acceptors is done using RMI. The sending of a command to the relayer and receiving of the command in the listening phase from the relayer is done using the current IPC mechanism.

As in previous assignments, you will continue adding to your current solution. You will see how different consensus algorithms implemented using different RPC mechanisms can affect local execution times as well as the nature of inconsistencies in objects replicated in client processes.

The entire assignment is extra credit, so ignore the division of tests into regular and extra credit.

Atomic and Non-Atomic Broadcast Modes of Input Commands

In all of the assignments leading up to this one, you have implemented the broadcast of commands using the non-atomic semantics. You will now get the chance to implement the alternative, atomic semantics. In non-atomic broadcast, clients can execute commands in different orders while in atomic broadcast, they always execute them in the same order. Non-atomic broadcast can cause inconsistencies if two simulations execute non-commuting commands (such as move and take) concurrently, that is, each simulation executes its local command before executing the remote command. Such inconsistencies do not occur if the two concurrent commands commute, that is, if the order in which they are executed does not matter. They are not created, for example, if the concurrent commands both move the simulation avatar as moves are additive (as long as the moves do not run into a border).

In the non-atomic broadcast you have implemented so far, a process executes an operation in the local *connected simulation* at its site and sends it to the server, which does not echo back the command to the process. In contrast, with atomic broadcast, a process sends the command, as a writer, to the server without executing it in the local

simulation, which is now a *disconnected simulation*. In this mode, the server echoes the command back to the local site. In both cases, all commands received from the server are executed on the learning local simulation – in the second case the received commands include local commands while in the first case it does not. As before, a receiving simulation uses the `processCommand()` method of the (connected/disconnected) simulation command processor to programmatically change its state, which does not fire events.

In the atomic mode, the client must invoke on the command processor the `setConnectedToSimulation(false)` to disconnect the command processor from the simulation, that is, to ensure it does not automatically forward the command to the local simulation. In the non-atomic mode, it must set the ConnectedToSimulation property of the command processor back to its default value of true.

Atomic and Non-Atomic Parameter and its Broadcast

The (A)tomicBroadcast dynamic client/server input parameter determines if broadcast of input commands is atomic. When the user changes this parameter in the server/client, the atomicBroadcast method is called in the server/client"

This is the template implementation:

```
@Override
/**
 * Relevant in consistency assignments only
 */
public void atomicBroadcast (boolean newValue) {
	super.atomicBroadcast(newValue);
	commandProcessor1.setConnectedToSimulation(!isAtomicBroadcast());
	commandProcessor2.setConnectedToSimulation(!isAtomicBroadcast());
}
```

As in the case of other user interactive parameters, setter and getter (`setAtomicBroadcast()` and `isAtomicBroadcast()`) methods in the AnAbstractSimulationParametersBean class you inherit are available to write and read this parameter.

Broadcast Meta State Dynamic Parameter

The AtomicBroadcast parameter, like the IPC mechanism, is part of the meta state. If the BroadcastMetaState dynamic parameter is true, the values of both the IPCMechanism and AtomicBroadcast parameters should be broadcast to all other clients. As before, a receiver processes all changes it receives and RMI is used for the broadcast.

So far you have done broadcast of meta state non-atomically using RMI. To reduce your implementation effort, we allow you to retain your current non atomic broadcast of meta state.

Stronger Consensus for Meta State (Optional)

Non-atomic broadcast of meta state is not ideal. Changes to AtomicBroadcast and IPCMechanism do not commute. It is also possible that not every site can implement each IPCMechanism – for instance GIPC may not be available on some computer. So a site should be able to veto a proposed change to the IPCMechanism.

You are encouraged to broadcast (using RMI) meta state atomically and use the consensus parameter described below to determine if it is broadcast using a two-phase commit (Centralized Synchronous).

IPC Mechanism and Stronger Consensus

So far you have implemented centralized asynchronous consensus above input commands using three different IPC Mechanisms, RMI, GIPC, and NIO.

Based on the discussion above, implement the stronger consensus – atomic centralized-asynchronous and centralized-synchronous - always using RMI, regardless of the current IPC mechanism. This means all the steps the server takes to achieve stronger consensus are communicated using RMI

Centralized Two-Phase Synchronous Consensus of Input Commands

Algorithm and Tracing

Recall that consensus is the problem of a set of communicating processes, C, agreeing on a value, V, of some replicated object, O. In all forms of consensus, some process proposes a new value of object named O. How other processes react to this proposal varies according to the form of consensus supported. The centralized asynchronous consensus algorithm you have implemented so far involved one round or phase of communication between the server and clients, in which the server simply informed the clients about the new value. The centralized synchronous consensus you will implement involves two phases. In the first phase, the server informs each *accepting* client about the new value and checks with it if the value is acceptable. In the second phase, it informs the clients about the new value if there is unanimous acceptance. Below are the steps in this form of consensus, some of which are common with centralized asynchronous consensus. For the data values communicated in this algorithm during the accept phase, they use the following enumeration in package *util.trace.port.consensus*:

```
public enum ProposalFeedbackKind {
	SUCCESS,
	SERVICE_FAULT,
	ACCESS_DENIAL,
	SERVICE_DENIAL,
	CONCURRENCY_CONFLICT,
	AGGREGATE_DENIAL // consolidated response
}
```

The new steps are italicized:

1. ProposalMade: A proposer makes proposal V for object O.
2. RemoteProposeRequestSent: The proposer sends a request to change O to a central server, S using the current IPC mechanism,
3. RemoteProposeRequestReceived: The server receives the request using the current IPC mechamism.
4. *ProposalAcceptRequestSent*: The server sends a query with O and V to each client and waits for a ProposalFeedbackKind `SUCCESS` or `ACCESS_DENIAL` response indicating if the client accepts the value. As you are using RMI to send the request, you will invoke an "accept" callback method in the client with these two parameters that returns a ProposalFeedbackKind value.
5. *ProposalAcceptRequestReceived*: The client receives the accept request and decides if it accepts the change or not. In either case, it invalidates the value of V by making it null. The learn notification will decide what the final value is.
6. *ProposalAcceptedNotificationSent*: The accepting client sends back a message to the server indicating if the value was accepted or not. In RMI, this means it simply traces this step before returning the result in the accept method.
7. *ProposalAcceptedNotificationReceived*: The server receives the ProposalFeedbackKind value from each client. Again, this means, tracing this step after receiving the return value
8. ProposalLearnedNotificationSent: The server uses the current IPC mechanism to send the new value to all of its clients. This is the new value if all clients accepted, otherwise it is null.
9. *ProposalLearnedNotificationReceived*: The receiving processes receive the value.
10. ProposedStateSet: The receiving processes change their local, replicated, copy to the received value, if this value is non-null. If the value is null, (a) if the object is a meta state, then it sets the value to the old value of the state, and (b) if the object is a command, it sets the value to the empty string. Changes to the command object result in the command being input to the simulation.

As in the previous assignment, to make you more aware of what you are doing and to automatically grade your assignment, you must trace these steps, using classes in the GIPC package, util.trace.port.consensus.

Again, to reduce your implementation effort, you are not *required* to detect concurrency conflicts. Thus, if a command arrives while another is pending, it is not rejected and instead executed after the previous one.

For ease of implementation. consensus can be done in RMI.

For fun, but no credit, you can do one of the following:

1. Make the relayer reject a proposal if it is concurrent with another concurrent proposal.
2. Make an acceptor reject a proposal if it is concurrent with another concurrent proposal,

Make an acceptor reject a proposal if it conflicts (does not commute) with an ongoing proposal.
Hint: You do not need the complete command history of proposals to determine if they are concurrent.

Handling Null Values of Meta State (Optional, Not graded)

An unaccepted value V of object O results in the changed object being null for a while. The null value of an object is an indication to the user of the object that the value of the object is not stable. Any action that depends on O should not be accepted. In particular, in case you implement a two-phase commit of meta state, if the IPC mechanism is null, then a new user command should not be broadcast. Instead, the newCase() method of the Consensus trace class ActionWhileEnablingProposalIsPending should be called to indicate that communication has been disabled while waiting for consensus.

Choosing the Consensus Algorithm

Whether centralized synchronous or asynchronous consensus is used for atomic broadcast is decided by the enum user parameter, (C)onsensusAlgorithm, whose values can be ***CENTRALIZED_ASYNCHRONOUS***, ***CENTRALIZED_SYNCHRONOUS,*** and several others with which we will not concern ourselves right now. The client and server values for this parameter are not required to be automatically synchronized – the end user can explicitly coordinate them. There is no tracing required when the consensus algorithm is changed.

Broadcasting them raises the issue of which consensus algorithm is used for this broadcast. If you wish, you can choose an arbitrary algorithm to keep them consistent.

The following inherited method is called when this parameter is set:

```
@Override
public void consensusAlgorithm(ConsensusAlgorithm newValue) {
	setConsensusAlgorithm(newValue);
}
```

Like other user interactive parameters, setter and getter methods (`setConsensusAlgorithm()` and `getConsensusAlgorithm()`) in the AnAbstractSimulationParametersBean class you inherit can be used to write and read this parameter.

Centralized synchronous consensus is by definition atomic as the sender waits for the value to be accepted by others before changing its state. Thus, the AtomicBroadcast parameter is relevant only when ConsensusAlgorithm is ***CENTRALIZED_ASYNCHRONOUS.***

Ideally, the `setConsensusAlgorithm()` method should automatically set AtomicBroadcast to true if its argument is ***`CENTRALIZED_SYNCHRONOUS`***. You may want to override the inherited implementation of this method to do so.

Choosing the Accept Response (Optional, Not Graded)

If you implement a two-phase commit of meta state, whether a client accepts a proposal to change the meta-state is determined by the user Boolean parameter, rejectMetaStateChange, found in AnAbstractSimulationParametersBean. Thus, acceptance is independent of the actual value proposed.

The following inherited method is called when this parameter is set:

```
public void rejectMetaStateChange(boolean newValue) {
            setRejectMetaStateChange(newValue);
}
```

User commands can always be accepted, that is, the response to accept is always SUCCESS when the property that is being changed is a command such as move or take.

However, as mentioned above, an acceptor may reject the command if it is in an ongoing accept phase determined by the pending status.

Example

Here is a set of traces to illustrate some of the required functionality, with the first set of traces identical to the one in a previous assignment.

Client Alice sets broadcast meta state to true and then changes the IPC mechanism (this is what was done in the previous assignment):

Alice:

```
Enter (prefixes of) print, quit or a method and its parameters
separated by commas:
b true
public default void
assignments.util.inputParameters.SimulationParametersListener.broadcast
MetaState(boolean)[true]

Enter (prefixes of) print, quit or a method and its parameters
separated by commas:
i GIPC
public default void
assignments.util.inputParameters.SimulationParametersListener.ipcMechan
ism(util.interactiveMethodInvocation.IPCMechanism)[GIPC]

I***@1100828022138418 {main}(RemoteProposeRequestSent)
EvtSrc(Your_Client_Simulation_Class_Name)  ipc_mechanism,-1.0=GIPC
```

```
I***@1100828051026069 {RMI TCP Connection(2)-
192.168.56.1}(ProposalLearnedNotificationReceived)
EvtSrc(Your_RMI_Relayer_Class_Name)  ipc_mechanism,-1.0=GIPC

I***@1100828051998227 {RMI TCP Connection(2)-
192.168.56.1}(ProposedStateSet)
EvtSrc(Your_Client_Simulation_Class_Name)  ipc_mechanism,-1.0=GIPC
Enter (prefixes of) print, quit or a method and its parameters
separated by commas:
```

As before, the first command results in no communication. In my implementation, the second one results in an **atomic** broadcast using RMI of the changed meta-state using the default centralized asynchronous communication mechanism. Thus, only learn notifications are received from the server. As mentioned earlier, you are free to use non atomic broadcasts.

Both the server and Alice change the consensus algorithm as my implementation does not keep them automatically consistent.

Alice:

```
Enter (prefixes of) print, quit or a method and its parameters
separated by commas:
c CENTRALIZED_SYNCHRONOUS
public default void
assignments.util.inputParameters.SimulationParametersListener.consensus
Algorithm(util.interactiveMethodInvocation.ConsensusAlgorithm)[CENTRALI
ZED_SYNCHRONOUS]

Enter (prefixes of) print, quit or a method and its parameters
separated by commas:
```

Server:

```
c CENTRALIZED_SYNCHRONOUS
public default void
assignments.util.inputParameters.SimulationParametersListener.consensus
Algorithm(util.interactiveMethodInvocation.ConsensusAlgorithm)[CENTRALI
ZED_SYNCHRONOUS]
```

No communication takes place in response to these two commands.

Alice now enters a command:

Alice:

```
s "move 100 -100"
```

1. ```
   public default void
   assignments.util.inputParameters.SimulationParametersListener.simula
   tionCommand(java.lang.String)[move 100 -100]
   ```
2. ```
   I***@1100903921476951 {main}(SetProperty)
   EvtSrc(AHalloweenCommandProcessor)  InputString = move 100 -100
   ```

3. I***@1100903922777534 {main}(CommandSubmitted)
   EvtSrc(AHalloweenCommandProcessor) Command:move 100 -100
4. I***@1100903974586461 {main}(NotifiedPropertyChangeEvent)
   EvtSrc(AHalloweenCommandProcessor) java.beans.PropertyChangeEvent[p
   ropertyName=InputString; oldValue=; newValue=move 100 -100;
   propagationId=null;
   source=stringProcessors.AHalloweenCommandProcessor@4df5bcb4]-
   >[nio.manager.simulation.client.AnNIOManagerSimulationClientOutCoupl
   er@cb51256,
   rmi.simulation.client.AnRMISimulationOutCoupler@4c371370,
   rmi.simulation.client.AnRMISimulationOutCoupler@126945f9]
5. I***@1100903975906431 {main}(RemoteProposeRequestSent)
   EvtSrc(AnRMISimulationOutCoupler) command,-1.0=move 100 -100
6. I***@1100927266102339 {Asynchronous Received Call
   Invoker}(ReceivedCallInitiated)
   EvtSrc(ADuplexReceivedCallInvoker) Default Server--
   >(Your_RMI_Relayer_Class_Name).public abstract boolean
   Your_RMI_Relayer_Class_Name.acceptNewCommand(java.lang.String)
   throws java.rmi.RemoteException([move 100 -100])
7. I***@1100927279724534 {Asynchronous Received Call
   Invoker}(ProposalAcceptRequestReceived)
   EvtSrc(Your_RMI_Relayer_Class_Name) Default Server--> command,-
   1.0=move 100 -100
8. I***@1100927280953274 {Asynchronous Received Call
   Invoker}(ProposalAcceptedNotificationSent)
   EvtSrc(Your_RMI_Relayer_Class_Name) command,-1.0=move 100 -100--
   >SUCCESS
9. I***@1100941847426402 {Asynchronous Received Call
   Invoker}(ReceivedCallInitiated)
   EvtSrc(ADuplexReceivedCallInvoker) Default Server--
   >(Your_RMI_Relayer_Class_Name).public abstract void
   rmi.simulation.common.RMISimulationInCoupler.processCommand(java.lan
   g.String) throws java.rmi.RemoteException([move 100 -100])
10. I***@1100941850731741 {Asynchronous Received Call
    Invoker}(ProposalLearnedNotificationReceived)
    EvtSrc(Your_RMI_Relayer_Class_Name) Default Server-->
    broadcast_mode,-1.0=move 100 -100
11. I***@1100942099946460 {Asynchronous Received Call
    Invoker}(CommandSubmitted)
    EvtSrc(AHalloweenCommandProcessor) Command:move 100 -100

Again, accept requests and notifications are sent using RMI but GIPC, the current IPC mechanism, is used for command communication – sending of commands to server and receiving of commands from it.,

Tracing

Use the new traces identified above. There are no new Factory.setTracing() methods to use setTracing() calls from the previous assignments are sufficient here.

Tags

These are the tags for this assignment. You need to add: `@DistributedTags.ATOMIC_BROADCAST` and `@DistributedTags.TWO_PHASE_COMMIT` to some of your existing classes.

Thus, you should have:

Edited:

```
@DistributedTags.CLIENT_CONFIGURER+@DistributedTags.RMI+@DistributedTags.GIPC+
@DistributedTags.ATOMIC_BROADCAST+@DistributedTags.TWO_PHASE_COMMIT

@DistributedTags.RMI+@DistributedTags.GIPC+@DistributedTags.NIO+@DistributedTa
gs.CLIENT+@DistributedTags.ATOMIC_BROADCAST+@DistributedTags.TWO_PHASE_COMMIT

@DistributedTags.CLIENT_REMOTE_OBJECT+@DistributedTags.RMI+@Distributed
Tags.GIPC+@DistributedTags.ATOMIC_BROADCAST+@DistributedTags.TWO_PHASE_
COMMIT,

@DistributedTags.SERVER_CONFIGURER+@DistributedTags.RMI+@DistributedTag
s.GIPC+@DistributedTags.NIO+@DistributedTags.ATOMIC_BROADCAST+@Distribu
tedTags.TWO_PHASE_COMMIT,

@DistributedTags.SERVER_REMOTE_OBJECT+@DistributedTags.RMI+@Distributed
Tags.GIPC+@DistributedTags.ATOMIC_BROADCAST+@DistributedTags.TWO_PHASE_
COMMIT,
```

Unchanged:

```
      @DistributedTags.CLIENT_OUT_COUPLER+@DistributedTags.RMI+@Distrib
utedTags.GIPC+@DistributedTags.NIO,

      @DistributedTags.SERVER_REMOTE_INTERFACE+@DistributedTags.RMI+@Di
stributedTags.GIPC,

@DistributedTags.REGISTRY+@DistributedTags.RMI,

      @DistributedTags.CLIENT_REMOTE_INTERFACE+@DistributedTags.RMI,


      @DistributedTags.CLIENT_REMOTE_INTERFACE+@DistributedTags.GIPC,

      @DistributedTags.RMI+@DistributedTags.GIPC+@DistributedTags.NIO+@
DistributedTags.SERVER,
```

Submission Instructions

Upload your code to Sakai and Gradescope, just as before. Submit screenshots of localchecks, and the console output reflecting atomic/non-atomic broadcasts, centralized

two-phase synchronous consensus, and any optional timing experiments with different ipc mechanisms, broadcast modes, and consensus types

Your screenshots should involve three clients and a server.

For Fun but Not Graded

Atomic versus Non-Atomic

(In) Consistency: Provide concurrent non-commuting commands (using breakpoints in the server or client) to two clients and show that atomic-broadcast and non-atomic broadcast result in consistent and inconsistent simulation replicas. The inconsistencies shown should be permanent, they should not be transient inconsistencies that go away when you release the breakpoints. You should change the broadcast mode parameter in each process manually by interacting with the user-interface. Recall that your program now has the ability to process a change to the broadcast mode in one client by broadcasting the new mode to all other processes, each of which then changes the mode.

Broadcast Mode Consensus with Multiple Clients

Ensure that the broadcast-mode is broadcast using two-phase synchronous consensus. Show the following sequence of steps:

- A proposing client changes the broadcast mode from atomic to non-atomic.
- Each of the accepting clients in the simulation breaks in the call that receives the consensus information.
- One or more simulation commands are input by users at the two accepting clients, which are ignored while the clients are blocked. This is because both of the clients will have received the proposal from the server, changed the value of their respective meta state objects to null, and sent a notification of their acceptance back to the server. Input commands are ignored when object values are null. There should be some visible feedback indicating that a command was ignored.
- Each client resumes execution.
- One or more simulation commands are input by users of at least two accepting clients, which are executed (along with possibly ignored commands).

You are free to use the delay parameters to show these effects. You can submit a video link rather than screenshots to show this.

Timing

Optional timing screenshots for this assignment should include timing the execution of 500 input commands at the site issuing these commands under the following 7 conditions (2 types of IPC, 2 types of broadcast, 2 types of consensus, and local processing by itself):

- Local execution only: the simulation executes the commands locally without sending any messages to remote sites.
- Basic Distributed 3-User RMI, Atomic Broadcast, Asynchronous Consensus: Same as above except that commands are broadcast to two other simulations using RMI with the atomic semantics and asynchronous consensus
- Basic Distributed 3-User GIPC, Atomic Broadcast, Asynchronous Consensus: Same as above except that GIPC is used for timing.
- Basic Distributed 3-User RMI, Atomic Broadcast, Two-Phase Synchronous Consensus: Same as above except that commands are broadcast to two other simulations using RMI and two-phase synchronous consensus
- Basic Distributed 3-User GIPC, Atomic Broadcast, Two-Phase Synchronous Consensus: Same as above except that GIPC is used for timing.
- Basic Distributed 3-User RMI, Non-Atomic Broadcast, Asynchronous Consensus: Same as above except that commands are broadcast to two other simulations using RMI with the non-atomic semantics and asynchronous consensus
- Basic Distributed 3-User GIPC, Non-Atomic Broadcast, Asynchronous Consensus: Same as above except that GIPC is used for timing.

Key Components of Implementation

Consult this section only if you are stuck trying to implement some aspect of the assignment. You will, of course, learn more by figuring out the implementation structure on your own based on the material above.

Centralized Synchronous Commit: Remote Objects

Your remote server and client objects now have additional methods. Do not add them directly to the existing interfaces and classes of these objects so that you can still have an independent A6 implementation. Instead extend the existing classes and interfaces.

Centralized Synchronous Commit: Sending-Observing Objects

The class of the observing-sending client object does not change, except it is passed a different proxy server object by the launchers/setup client program.

Centralized Synchronous Commit: Client and Server Launchers/Setup

Your RMI server launcher will now have to register with the RMI registry an object of a different class *instead* of (and not in addition to) the one registered in the previous assignment – a user parameter in the server decides if it calls the superclass methods (centralized asynchronous commit) or the subclass ones (centralized synchronous commit). You can make the new RMI server launcher a subclass of the A6 launcher. To reuse the code from the superclass, define a factory method in the A6 server launcher that instantiates the server object.

Your client launcher will have to join the server session with a different remote client object. Again, use subclassing and factory methods to create the new client launcher class. You should not have to change the code that passes to the observing-sending object

the server proxy – the same name will be used to retrieve an object of a new interface, which is a subtype of the previous one.

Possible Coding Issues

1. Not using non-commuting commands to create/not create inconsistency with non-atomic/atomic /broadcast.
2. What happens if you go from a weaker level of consistency (local < broadcast<atomic broadcast) to a stronger level
    a. The change applies to future and not past messages.
3. What if the server and client have set different levels of consistency?
    a. In our tests and yours, the user of each inputting client and the server should make the same change.
4. Can we automatically sync the broadcast modes
    a. Yes, this is an optional feature
5. Should the timing results determine how long it took to process the commands in the client
    a. No, in atomic broadcast, the performance experiments will end before the client has received the commands. The experiment measures the time taken by the main thread to write to the socket channel.
    b. Make sure you have not included GIPC or Beau's project in your submission.
6. Make sure that phase one of synchronous consensus is done only with RMI regardless of what IPC mechanism (GIPC/RMI) is used.

GPT Summary (Not checked for correctness)

Can you give a summary of the key points of this assignment?

ChatGPT said:

Here is a **summary of the key points** of **Assignment 7: Centralized Two-Phase Synchronous Consensus and Atomic Broadcast** (Comp 533):

🎯 Assignment Goal

Introduce **atomic broadcast** and implement **centralized two-phase synchronous consensus** for achieving stronger agreement guarantees among clients in a distributed system. This is an **extra credit assignment** and extends your Assignment 6 solution using **RMI** (only) for consensus operations.

🔁 Broadcast Modes

1. Non-Atomic Broadcast (default in previous assignments):

- Clients **execute their own commands immediately** and send them to the server.
- Server broadcasts to other clients (excluding the sender).
- Can result in **inconsistent state** if commands do not **commute**.

2. Atomic Broadcast (new in this assignment):

- Clients **do not immediately execute their own commands**.
- All commands (including local) are echoed back from the server and then executed **in the same order everywhere**.
- Requires commandProcessor.setConnectedToSimulation(false) in atomic mode.

Meta-State Parameters

- atomicBroadcast(boolean) — dynamic flag controlling broadcast mode (default: non-atomic).
- broadcastMetaState(boolean) — if true, changes to ipcMechanism and atomicBroadcast are broadcast to other clients via **RMI**.

Stronger Consensus: Centralized Two-Phase Synchronous Commit

Two-phase protocol:

All communication uses RMI, regardless of IPC mode.

Phases:

1. **Accept phase**:
    - Server proposes value V of object O to clients.
    - Each client responds with a ProposalFeedbackKind (e.g., SUCCESS, ACCESS_DENIAL, etc.)
2. **Learn phase**:
    - If all clients accept, server broadcasts the value to all clients.

Tracing Events (must be logged):

- ProposalMade, ProposalAcceptRequestSent/Received
- ProposalAcceptedNotificationSent/Received
- ProposalLearnedNotificationSent/Received
- ProposedStateSet

Null value behavior:

- If consensus fails:
    - Meta-state reverts to previous value.
    - Command becomes empty string (ignored).

- Optional trace: ActionWhileEnablingProposalIsPending (when actions are blocked due to unstable state).

⚖️ Consensus Algorithm Selection

- User parameter: consensusAlgorithm(ConsensusAlgorithm)
  - Options: CENTRALIZED_ASYNCHRONOUS, CENTRALIZED_SYNCHRONOUS
- If CENTRALIZED_SYNCHRONOUS, automatically set atomicBroadcast(true) (recommended).
- Meta-state consensus is optional; you can broadcast changes non-atomically for simplicity.

🧪 Performance Experiment (Optional)

Run timing comparisons under 7 modes:

1. **Local-only**
2. **RMI + Atomic + Async Consensus**
3. **GIPC + Atomic + Async**
4. **RMI + Atomic + Two-Phase Sync**
5. **GIPC + Atomic + Two-Phase Sync**
6. **RMI + Non-Atomic + Async**
7. **GIPC + Non-Atomic + Async**

Use the experimentInput() and localProcessingOnly(boolean) APIs already integrated in your earlier assignments.

🏷️ Updated Tags Required

Apply these to relevant classes:

- @DistributedTags.ATOMIC_BROADCAST
- @DistributedTags.TWO_PHASE_COMMIT

Update existing classes (e.g., CLIENT, SERVER, CONFIGURER, REMOTE_OBJECT) to include these.

🏗️ Structural Notes

- Use subclassing to avoid breaking Assignment 6 implementations.
- Add new accept(...) methods to **new interfaces** extending the old ones.
- Use factory methods in launchers to create appropriate objects.

⚠️ Common Pitfalls

- Forgetting to trace all consensus events.
- Not handling null values after failed consensus.
- Blocking on a proposal or executing commands during pending consensus.
- Forgetting to disable simulation linkage in atomic mode.
- Not updating client/server consensus modes to match.
- Not forcing RMI for consensus (using GIPC by mistake).

Would you like a checklist version of this summary, or should I generate a template to help you structure the two-phase consensus code and tracing?

2/2

4o

You said:

yes

ChatGPT said:

Here is a structured ✅ **checklist version** of the **Assignment 7** requirements, followed by a 🧩 **template for implementing two-phase consensus** with the required tracing.

✅ Assignment 7 Implementation Checklist

🔁 Broadcast Modes

- Add support for **atomic broadcast**:
  - On atomicBroadcast(true), set commandProcessor.setConnectedToSimulation(false)
  - On atomicBroadcast(false), reset to true
- Implement dynamic broadcast mode switching:
  - Use atomicBroadcast(boolean newValue)
  - Use setAtomicBroadcast() and isAtomicBroadcast() methods

📡 Meta-State Broadcast (via RMI)

- Retain **non-atomic RMI-based broadcast** of ipcMechanism and atomicBroadcast
- Use broadcastMetaState(true) to enable propagation
- Optional: Broadcast meta-state **atomically** using two-phase consensus

---

🔐 Centralized Two-Phase Synchronous Consensus (via RMI)

Phase 1: Proposal Accept

- Server invokes client's accept(String objectName, String proposedValue) via RMI
- Client:
  - Sets value to null
  - Returns ProposalFeedbackKind.SUCCESS (or other)
  - Traces:
    - ProposalAcceptRequestReceived
    - ProposalAcceptedNotificationSent

Phase 2: Learn

- Server:
  - Collects all feedback
  - If all success: broadcasts proposal using **current IPC**
  - Else: broadcasts null
  - Traces:
    - ProposalLearnedNotificationSent
- Client:
  - Receives proposal or null
  - Applies value if not null
  - Traces:
    - ProposalLearnedNotificationReceived
    - ProposedStateSet

🧠 Consensus Algorithm Mode

- Support parameter consensusAlgorithm(...) with:
  - CENTRALIZED_ASYNCHRONOUS (1-phase, current default)
  - CENTRALIZED_SYNCHRONOUS (2-phase, new)
- Automatically set atomicBroadcast(true) if CENTRALIZED_SYNCHRONOUS is selected

---

🏷️ Required Tags

Apply to appropriate classes:

java

CopyEdit

```
@Tags({
  DistributedTags.RMI,
  DistributedTags.GIPC,
  DistributedTags.ATOMIC_BROADCAST,
  DistributedTags.TWO_PHASE_COMMIT,
  DistributedTags.CLIENT / SERVER / CONFIGURER / REMOTE_OBJECT
})
```

🧪 Optional Timing Experiments

For each of the 7 modes:

- Use l true and l false for local-only mode toggle
- Use e to trigger experiment with 500 commands
- Document results across:
    - Broadcast types (atomic vs non-atomic)
    - Consensus types (sync vs async)
    - IPC (RMI, GIPC)